



SECTION 1: PROJECT OBJECTIVE

Project Title

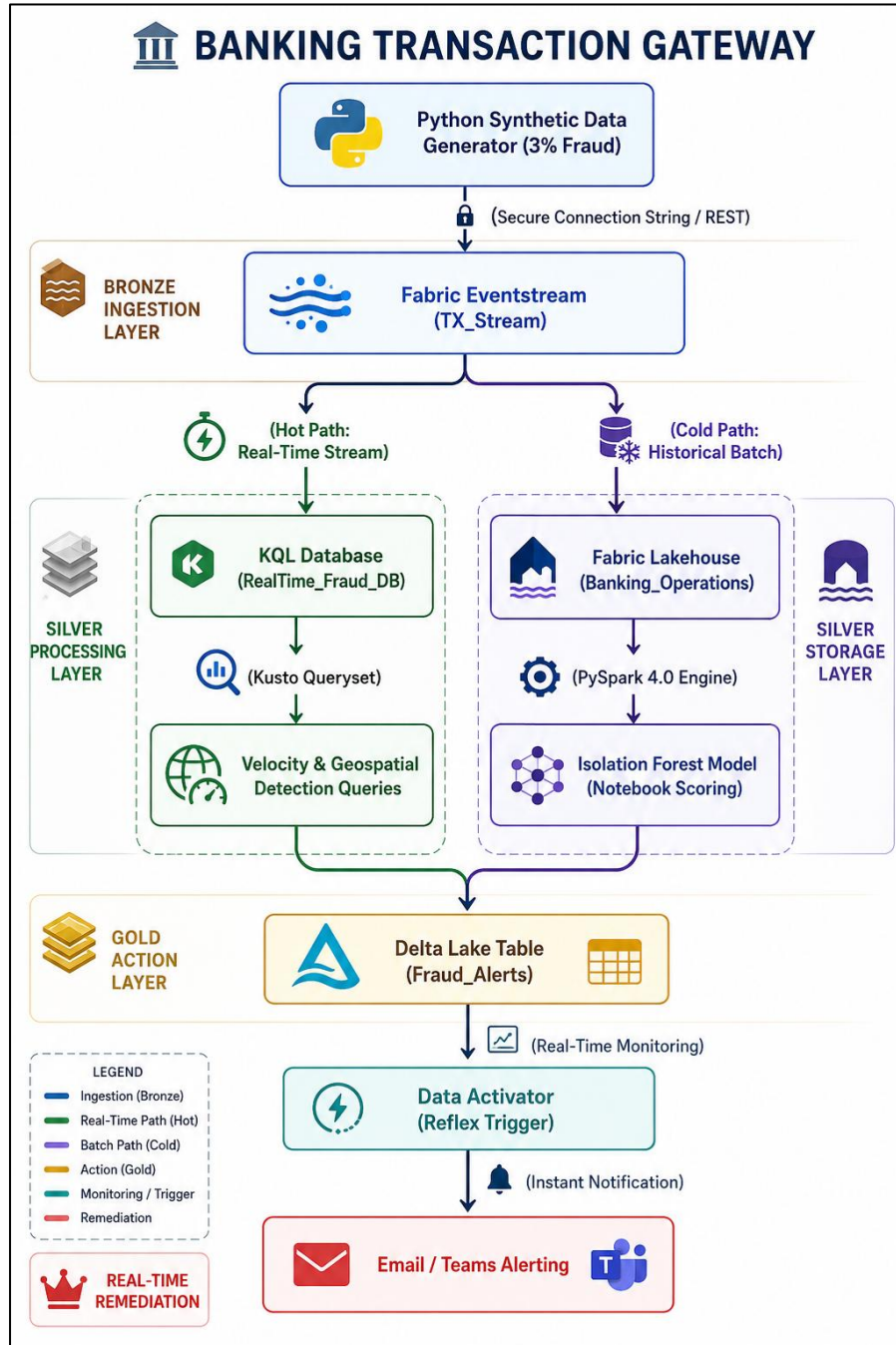
Architecting a Real-Time Banking Fraud Detection System in Microsoft Fabric

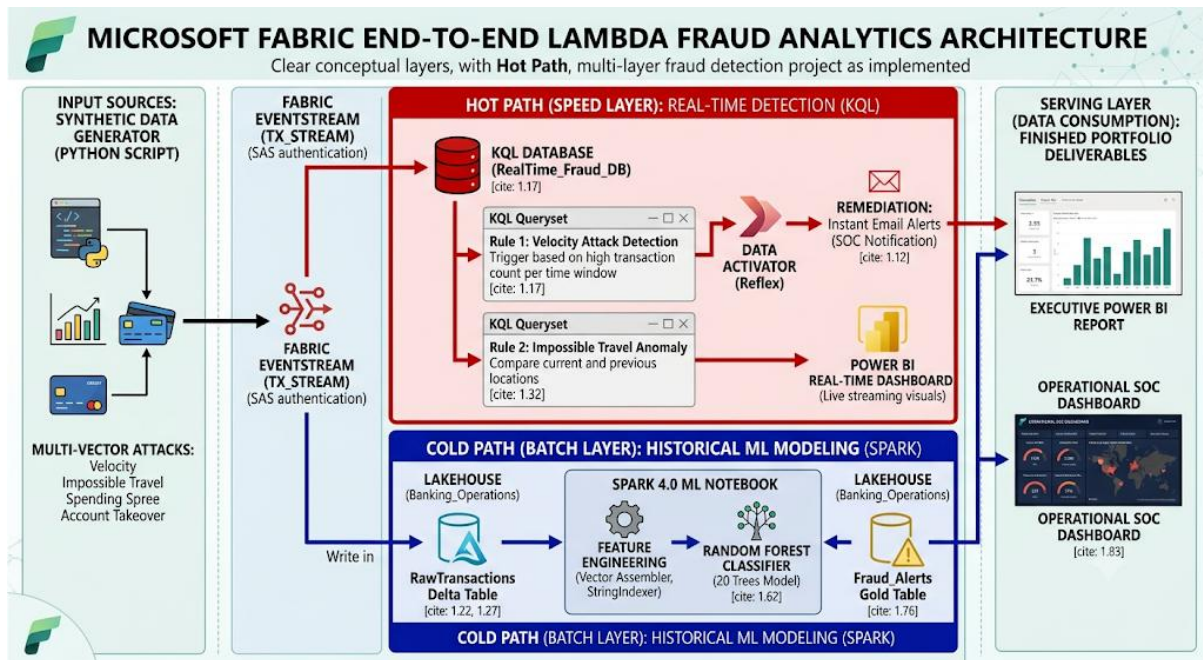
Objective & Scope

The objective of this hands-on project is to design, deploy, and evaluate an end-to-end, high-velocity data streaming architecture capable of detecting fraudulent banking transactions as they occur.

SECTION 2: SYSTEM ARCHITECTURE

This project implements a **Lambda-style architecture** split across two distinct paths (Hot and Cold) to balance real-time responsiveness with deep historical AI modeling.





The 4 Pipeline Stages

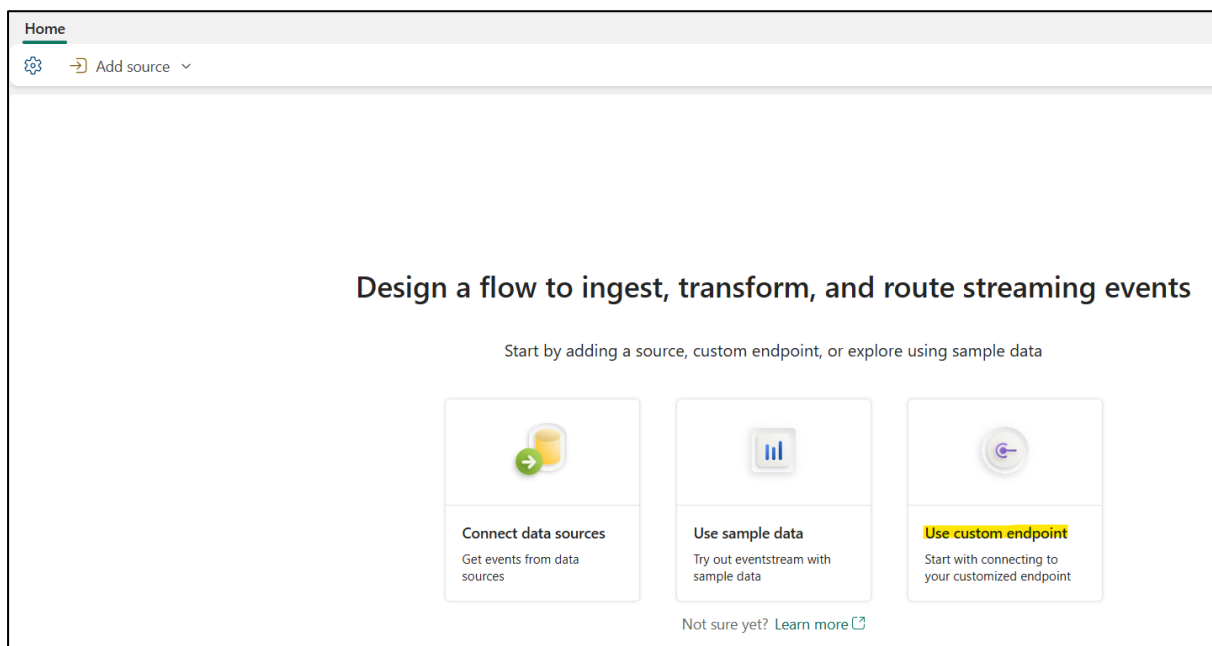
1. **Source:** A Python script simulates a live banking gateway, continuously streaming ATM and online transaction payloads.
2. **Ingestion (Bronze): Fabric Eventstream** captures the incoming data with high throughput and splits it into parallel branches.
3. **Processing & Storage (Silver):**
 - **Hot Path:** The stream flows into a **KQL Database** for immediate, low-latency querying via Kusto Query Language.
 - **Cold Path:** The stream simultaneously saves to a **Lakehouse**, where a **Spark** Notebook trains an Isolation Forest ML model to detect complex anomalies.
4. **Action (Gold):** Model predictions and critical KQL flags land in a final **Fraud_Alerts** destination. **Data Activator** monitors this layer to instantly trigger automated alerts (such as email or Teams warnings) when fraud is confirmed. Also Power BI report & Real Time dashboard to show key visualizations.

Step 3: Provisioning the Environment

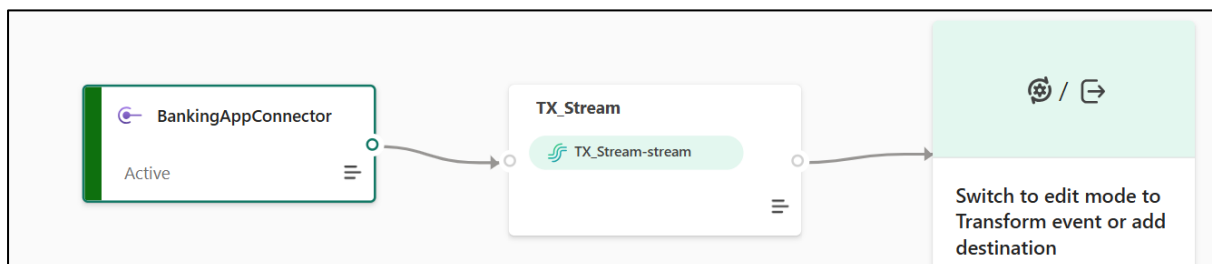
1. Log into the **Microsoft Fabric** web portal.
2. Create a workspace called Realtime_Fraud_Detection
3. Create a Lakehouse called Banking_Operations
4. Create a new Eventhouse called RealTime_Fraud_DB

Step 4: Setting up the Stream (Bronze Ingestion Layer)

- 1) Create a new Eventstream called **TX_Stream**



- 2) Click on Use custom endpoint. Provide the below details
 - a. **Source Name:** BankingAppConnector
- 3) Click Add
- 4) Click Publish button



- 5) Click on BankingAppConnector
- 6) Click SAS Key Authentication.

Details

Protocol:

Event Hub

AMQP

Kafka

Basic

Entra ID Authentication

SAS Key Authentication

Event hub name

Shared access key name

Primary key

Secondary key

Connection string-primary key

Connection string-secondary key

esehpn8wikj2uysgikyq_eh

key_6149519f-71da-4c5d-b83a-0e83e05018f7

XQdUoyOcXS4tp7rp35cd7PxEGDOy8SSBH+AEhMQC

hpqrglSmi4j/b/6toCQ7Agcda+MvUtRMZ+AEhK/B/Y

Endpoint=sb://esehpn8wikj2uysgikyq.servicebus.v

Endpoint=sb://esehpn8wikj2uysgikyq.servicebus.v

- 7) Copy values Event hub name and Connection string-primary key values to a notepad. Click on eye icon if not enabled.



Step 5: Data Generation (Python Simulation)

- 1) Open Python code editor
- 2) Install Azure Event Hubs library if not installed
pip install azure-eventhub
- 3) Open file Generate Transaction Data.py in the editor
- 4) Replace values CONN_STR & EH_NAME with the values copied in the previous step
- 5) Run the script
- 6) You must see the values in the Eventstream after few seconds

Data preview		Data insights					Refresh
Hide settings		Search					Show details
Data type Json		tx_id	cust_id	amount	location	timestamp	fraud_type
Last refreshed 06/23/26 10:12:42 PM		950482	USER_15	277.15	NYC	2026-06-23T16:42:31Z	None
		154977	USER_18	343.14	PAR	2026-06-23T16:42:30Z	None
		438441	USER_42	309.8	LDN	2026-06-23T16:42:29Z	None
		509688	USER_44	4331.31	High_Risk_Zone	2026-06-23T16:42:28Z	Velocity_Attack
		947927	USER_43	568.03	PAR	2026-06-23T16:42:28Z	None
		616246	USER_33	429.69	NYC	2026-06-23T16:42:27Z	None
		320003	USER_8	490.68	NYC	2026-06-23T16:42:26Z	None
		488705	USER_30	337.68	PAR	2026-06-23T16:42:26Z	None

How the Data Generator Works: This script acts as a live simulator of a busy banking network. Every half-second, it loops to create a new transaction payload containing essential details like a unique transaction ID, user ID, purchase amount, location, and a precise timestamp. To make the simulation realistic, the script uses a randomized "dice roll" system to blend normal everyday spending with **four distinct fraud patterns**:

- **Velocity Attacks:** Mimics a stolen card being rapidly swiped for high dollar amounts.
- **Impossible Travel:** Simulates a card appearing in completely different global cities (like Tokyo then Mumbai) faster than a human could physically fly.
- **Spending Sprees:** Injects sudden, extreme luxury purchases (\$12,000+) to test our AI model's outlier detection.
- **Account Takeovers:** Fires off rapid, tiny micro-charges under \$5.00 to see if a fraudster is secretly testing an active account.

Finally, the script packages this data into JSON format and uses a secure **SAS connection string** to stream the events directly into the Fabric Eventstream engine.

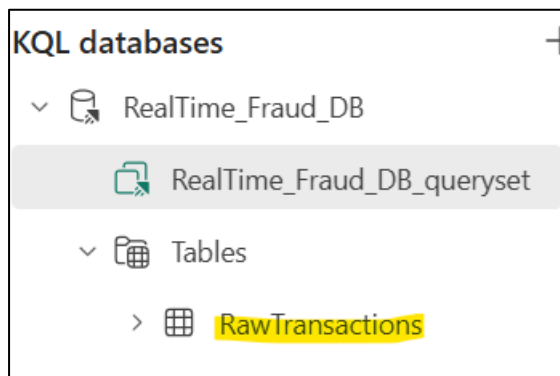
Step 6: Real-Time Analytics (KQL Hot Path)

Part A: Pre-Creating the KQL Table

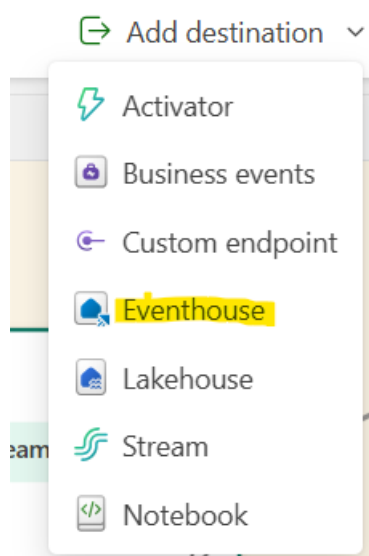
1. Go to the Eventhouse RealTime_Fraud_DB
2. Click RealTime_Fraud_DB_queryset
3. Execute the following script to create a table

```
.create table RawTransactions (  
    tx_id: long,  
    cust_id: string,  
    amount: real,  
    location: string,  
    timestamp: datetime,  
    fraud_type: string  
)
```

A new table RawTransactions is created



4. Return to your **Fabric Eventstream** canvas (TX_Stream)
5. Click **Add destination** on the top ribbon menu and select **Eventhouse**.



6. In the configuration panel on the right side:
 - **Data Ingestion mode:** Event processing before ingestion
 - **Destination name:** KQL_Transactions_Target
 - **Workspace:** *Select your active lab workspace*
 - **Eventhouse:** RealTime_Fraud_DB
 - **Target table:** Select RawTransactions
7. Click Save
8. Connect TX_Stream to destination if not connected
9. Click Publish

Eventhouse

This destination lets you ingest your real-time event data into an eventhouse, where you can use the powerful Kusto Query Language (KQL) to query and analyze the data.

Data ingestion mode *

☐ Direct ingestion

☒ Event processing before ingestion

This cannot be changed once this KQL destination starts ingesting.

Destination name *

KQL_Transactions_Target

Workspace *

Realtime_Fraud_Detection

Eventhouse *

RealTime_Fraud_DB

KQL Destination table *

(New) RawTransactions

Create new

Input data format * ⓘ

Json

☒ Activate ingestion after adding the data source

Save

Once the data is refreshed, should see the records in the table.

RealTime_Fraud_DB

System overview
Databases
Monitoring ⓘ

Search

KQL databases

RealTime_Fraud_DB

RealTime_Fraud_DB_q... ⓘ

Tables

RawTransactions

Ingestion Mappings
Update Policy

tx_id

Database / Tables / RawTransactions

Overview
Entity diagram ⓘ

Data Activity Tracker

Ingestion: 218 rows
Last run: 8:50:18 PM

218

0

Jun 21, 12:00 PM Jun 22, 12:00 AM Jun 22, 12:00 PM Jun 23, 12:00 AM Jun 23, 12:00 PM Jun 24, 12:00 AM Jun 24, 12:00 PM

Data preview
Schema insights

Search keyword

tx_id	cust_id	amount	location	timestamp	fraud_type	Ingestion
> 457940	USER_40	111.88	PAR	2026-06-24T15:19:14.000Z	None	2026
> 976163	USER_1	493.91	NYC	2026-06-24T15:19:15.000Z	None	2026
> 830359	USER_47	412.21	NYC	2026-06-24T15:19:15.000Z	None	2026
> 879004	USER_1	6555.29	High_Risk_Zone	2026-06-24T15:19:16.000Z	Velocity_Attack	2026
> 410129	USER_15	366.05	PAR	2026-06-24T15:19:17.000Z	None	2026
> 126539	USER_7	161.71	PAR	2026-06-24T15:19:17.000Z	None	2026

Top 100 records from the last 365 Days, limited by the hot cache retention

Part B: Authoring Fraud Detection Queries (KQL)

- Click on KQL queryset  KQL Queryset
- Provide the name as Fraud_Investigation_Queries

New KQL Queryset

Name *

Location *

 Realtime_Fraud_Detection

▼

↗

Create

Cancel

3. Remove all the queries
4. Copy and paste the two detection rules below into the editor.

Rule 1: Catching Velocity / Card Swiping Attacks

This rule calculates a rolling 10-second window for every customer. If an account logs more than 3 transactions within that tiny window, it immediately flags it as a high-velocity threat.

// Hot Path Rule 1: Flag high-frequency card swiping behavior

RawTransactions

| where amount > 0

| summarize tx_count = count(), total_spend = sum(amount) by cust_id, bin(timestamp, 10s)

| where tx_count > 3

| project timestamp, cust_id, tx_count, total_spend, AlertReason = "Velocity Attack Detected"

Rule 2: Catching Impossible Travel Anomalies

This query uses a stateful function (prev()) to compare a user's current transaction location against their very last transaction. If the location changes (e.g., from NYC to Tokyo) within less than 2 minutes (120 seconds), it triggers a fraud alert.

// Hot Path Rule 2: Flag geographic location conflicts within 2 minutes

RawTransactions

```
| order by cust_id asc, timestamp desc
```

```
| extend prev_location = prev(location), prev_time = prev(timestamp), prev_cust =  
prev(cust_id)
```

```
| where cust_id == prev_cust and location != prev_location
```

```
| extend time_difference_seconds = datetime_diff('second', prev_time, timestamp)
```

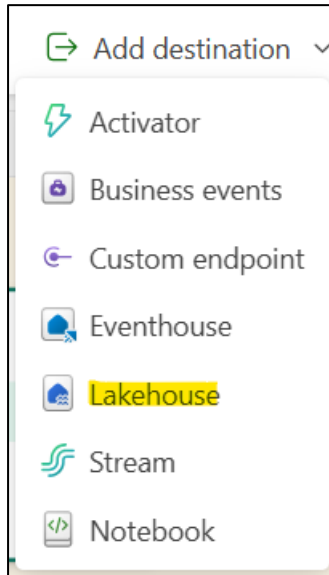
```
| where time_difference_seconds between (1 .. 120)
```

```
| project timestamp, cust_id, CurrentLocation = location, PreviousLocation = prev_location,  
time_difference_seconds, AlertReason = "Impossible Travel Anomaly"
```

Step 7: AI Model Deployment (Spark Cold Path)

Part A: Route Eventstream to the Lakehouse (Cold Path Archive)

1. Return to your **Fabric Eventstream** canvas (TX_Stream)
2. Click **Add destination** on the top ribbon and select **Lakehouse**.



3. In the configuration panel:
 - **Destination name:** Lakehouse_Delta_Target
 - **Workspace:** *Select your active workspace*
 - **Lakehouse:** Banking_Operations
 - **Target table:** Select **New table** and type RawTransactions and click Done.

Create new table

Table schema *

☒ Select an existing schema

☐ Create a new schema

dbo

Table name *

RawTransactions

Done **Cancel**

- Click **Save**.
- 4. Connect TX_Stream to destination if not connected
- 5. Click **Publish** on the top toolbar to begin writing a historical copy of the stream into your Lakehouse Delta table.
- 6. After sometime, you should be able to see the records in the lakehouse

Explorer

Add lakehouses

Banking_Operations

Tables

dbo

RawTransactions

Files

RawTransactions

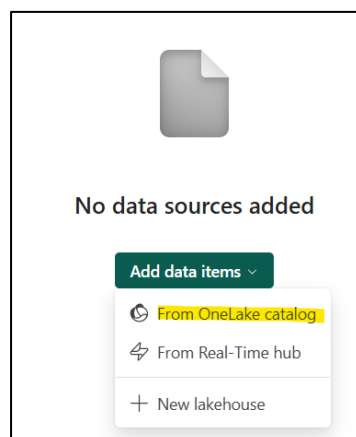
Table view

Showing 60 rows

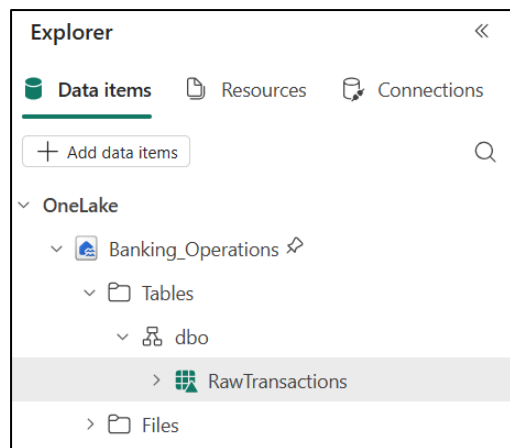
	tx_id	abc_cust_id	amount	abc_location	abc_timestamp	abc_fraud_type
1	249505	USER_44	221.84	MUM	6/24/2026 3:31:54 PM	Impossible_Travel
2	241751	USER_30	191.97	NYC	6/24/2026 3:31:56 PM	None
3	688816	USER_37	497.64	PAR	6/24/2026 3:31:58 PM	None
4	312744	USER_21	21906.15	LDN	6/24/2026 3:32:01 PM	Spending_Spree
5	659863	USER_5	55.14	NYC	6/24/2026 3:32:03 PM	None
6	464559	USER_29	569.36	LDN	6/24/2026 3:32:06 PM	None
7	969305	USER_18	377.2	LDN	6/24/2026 3:32:08 PM	None
8	660591	USER_25	143.22	LDN	6/24/2026 3:32:11 PM	None
9	134488	USER_14	165.18	NYC	6/24/2026 3:32:13 PM	None
10	543750	USER_31	598.26	PAR	6/24/2026 3:32:16 PM	None
11	878246	USER_18	212.68	PAR	6/24/2026 3:32:18 PM	None
12	855741	USER_31	596.6	LDN	6/24/2026 3:32:20 PM	None
13	171062	USER_13	209.49	LDN	6/24/2026 3:32:23 PM	None
14	267737	USER_27	13.41	LDN	6/24/2026 3:32:25 PM	None

Part B: Build and Run the ML Notebook

1. Go to workspace and create a new Notebook item
2. Provide the name as Fraud_Alerts_Analysis
3. Click Add data items
4. Select From Onelake catalog



5. Select the lakehouse and click Add



6. Copy paste the code in Fraud_Analysis.py in the first cell
7. Run the code
8. Once completed, should see a new table called fraud_alerts in the lakehouse

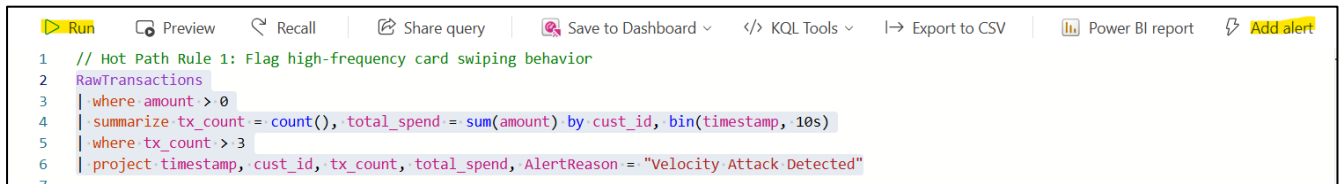
tx_id	cust_id	FlaggedAmount	location	timestamp	fraud_type
186210	USER_20	4.61	Online_Gateway	6/24/2026 3:32:49 PM	Account_Takeover
190010	USER_31	5521.59	High_Risk_Zone	6/24/2026 3:33:57 PM	Velocity_Attack
134281	USER_28	24139.76	PAR	6/24/2026 3:34:15 PM	Spending_Spree
194973	USER_1	4179.84	High_Risk_Zone	6/24/2026 3:36:44 PM	Velocity_Attack
135731	USER_46	2.26	Online_Gateway	6/24/2026 3:37:00 PM	Account_Takeover
134160	USER_23	285.08	TOK	6/24/2026 3:37:23 PM	Impossible_Travel
114161	USER_8	2.93	Online_Gateway	6/24/2026 3:37:26 PM	Account_Takeover
173632	USER_38	128.38	SYD	6/24/2026 3:37:34 PM	Impossible_Travel
164971	USER_9	18215.3	NYC	6/24/2026 3:37:38 PM	Spending_Spree
108811	USER_33	12868.32	PAR	6/24/2026 3:37:55 PM	Spending_Spree
149505	USER_44	221.84	MUM	6/24/2026 3:31:54 PM	Impossible_Travel
112744	USER_21	21906.15	LDN	6/24/2026 3:32:01 PM	Spending_Spree
137310	USER_29	1.05	Online_Gateway	6/24/2026 3:31:55 PM	Account_Takeover
125047	USER_3	3.92	Online_Gateway	6/24/2026 3:31:55 PM	Account_Takeover

Logic: Think of the **Random Forest** model as a committee of 20 virtual fraud investigators. Instead of looking at just one simple rule, this model looks at the whole story behind a transaction: how much money was spent and where the card was used. Because computers can only read numbers, the script first translates city names (like "NYC" or "Paris") into numeric codes. Then, it sends this data to all 20 investigators. Each investigator builds a custom "clue tree" to analyze how unusual the purchase looks compared to the customer's normal habits. Finally, the investigators take a majority vote. If they agree a transaction looks highly suspicious, the model instantly pulls it out of the main database and writes it to a special **Fraud_Alerts** warning list so the bank can take immediate action.

⚡ Step 8: Real-Time Remediation (Data Activator)

Part A: Linking the KQL Query to Data Activator

1. Open your **Fraud_Investigation_Queries** KQL Queryset
2. Highlight **Rule 1: Velocity Attack Detection** query text and run the query
3. Once query runs, look at the top menu bar of the Queryset window and click the **Add Alert** button

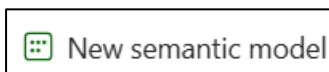


```
1 // Hot Path Rule 1: Flag high-frequency card swiping behavior
2 RawTransactions
3 | where amount > 0
4 | summarize tx_count = count(), total_spend = sum(amount) by cust_id, bin(timestamp, 10s)
5 | where tx_count > 3
6 | project timestamp, cust_id, tx_count, total_spend, AlertReason = "Velocity Attack Detected"
7
```

4. In the side panel that appears on the right, configure the following settings:
 - **Alert Name:** Velocity_Fraud_Alert_Trigger
 - **Run query every:** Set this to **5 minutes**
 - **Check:** On each event
 - **Action:** Select **Email**.
 - **Subject:** ⚠ CRITICAL CRIME ALERT: High Velocity Fraud Detected!
 - **Headline:** Urgent Account Freeze Required
 - **Message Context:** Click the **Add Field** button to dynamically inject the cust_id and the total_spend directly from the KQL query output into the email body text.
 - **New item name:** Velocity Fraud Alert Activator
5. Click **Create** to automatically provision a brand new Data Activator Reflex item in your workspace.
6. Repeat the steps for the other query also with suitable name and content.

Part B: Build a Power BI Executive Dashboard

1. Open your **Banking_Operations** Lakehouse
2. On the top toolbar menu, click the **New Semantic Model** button



3. A configuration window will pop up. Enter the following details:
 - **Name the model:** Gold_Fraud_Semantic_Model

- **Select the tables:** Check the boxes next to Fraud_Alerts and RawTransactions.

4. Click **Confirm**.

New semantic model

Direct Lake semantic model name

Workspace

Select or deselect tables for the semantic model.

SQL views cannot be selected for Direct Lake on OneLake, but tables based on SQL views can be added to the semantic model later using other storage modes.

☒ Select all

✓ ☒ dbo

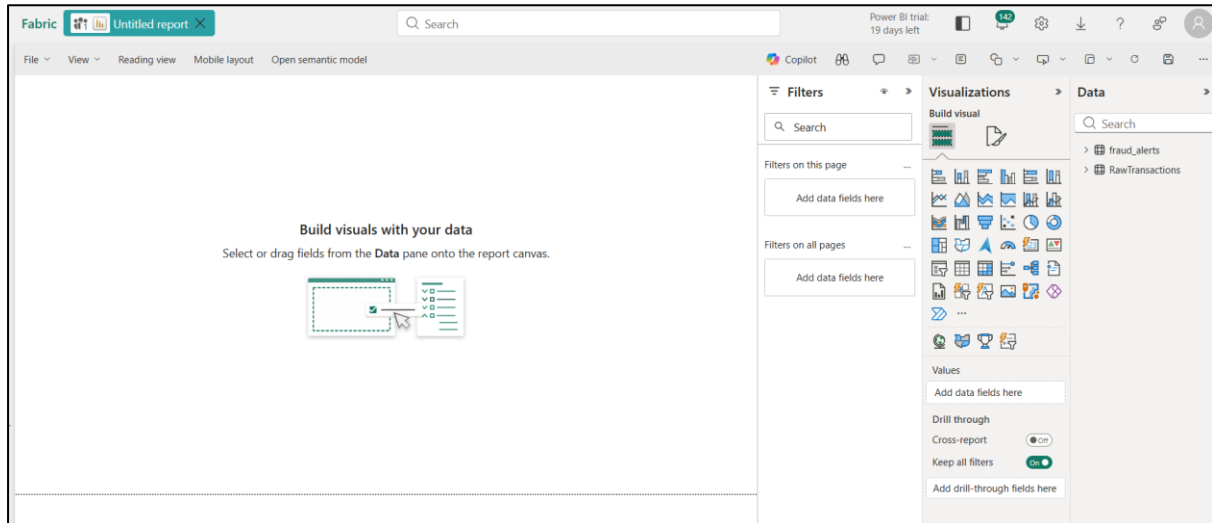
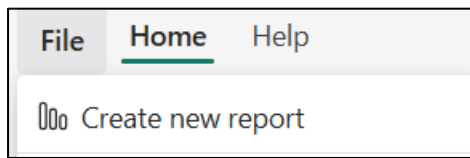
✓ ☒ fraud_alerts

✓ ☒ RawTransactions

[Learn more about Direct Lake](#) Confirm Cancel

5. Wait for the semantic model to open. If it does not open, go to workspace, click on the semantic model item and click on **Open** button. Change the mode to Editing

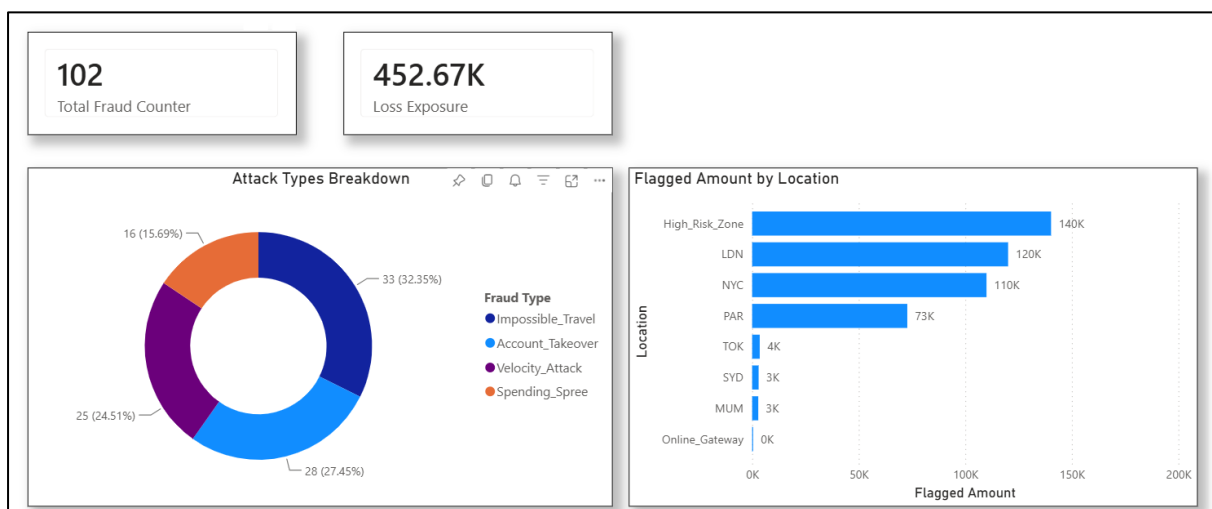
6. Click File -> Create new report. Power BI report window opens.



7. Build the following visuals. Feel free to add additional visuals.

Visual Type	Field	Aggregation	Title
Card	FlaggedAmount	Count	Total Fraud Counter
Card	FlaggedAmount	Sum	Loss Exposure
Donut	<i>Legend fraud_type</i> Values tx_id (Count)		Attack Types Breakdown
Bar chart	<i>Location</i> <i>FlaggedAmount</i>		Geographic Threat

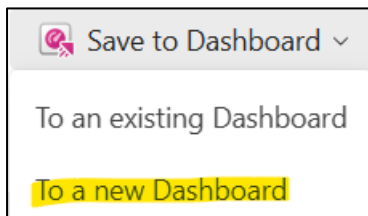
8. Save the report as Fraud_Operations_Dashboard



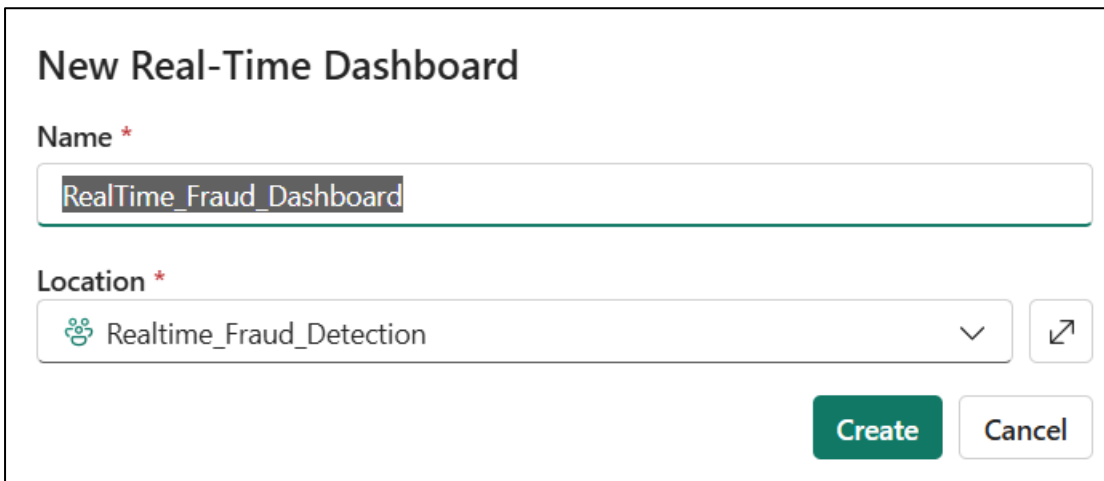
Step 9: Building the Real-Time Security Dashboard

Part A: Creating the Dashboard and Pinning Queries

1. Open your **Fraud_Investigation_Queries** KQL Queryset
2. Highlight the entire text block for **Rule 1: Velocity Attack Detection**
3. Click the **Save to dashboard** -> To a new Dashboard

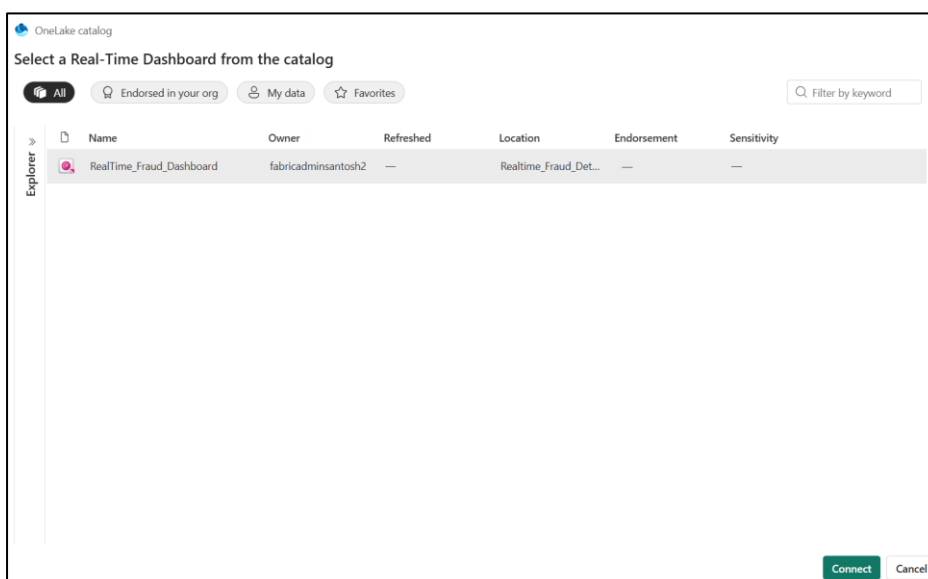


4. Provide dashboard name as `RealTime_Fraud_Dashboard` and click **Create**



A screenshot of the "New Real-Time Dashboard" form. The "Name" field is filled with "RealTime_Fraud_Dashboard". The "Location" dropdown is set to "Realtime_Fraud_Detection". There are "Create" and "Cancel" buttons at the bottom right.

5. Select second query and click **To an existing Dashboard**
6. Select `RealTime_Fraud_Dashboard` and click **Connect**



7. Click Open Dashboard

timestamp	cust_id	tx_count	total_spend
> 2026-06-24 15:33:30.000	USER_18	4	1,568.360000
> 2026-06-25 15:33:20.000	USER_17	4	

timestamp	cust_id	CurrentLocation	PreviousLocation
> 2026-06-25 15:45:00.000	USER_1	NYC	Hiç
> 2026-06-25 15:43:57.000	USER_1	PAR	NY
> 2026-06-25 15:32:42.000	USER_1	LDN	NY
> 2026-06-25 15:32:15.000	USER_1	High_Risk_Zone	LDI
> 2026-06-25 15:30:47.000	USER_1	PAR	Hiç
> 2026-06-25 15:29:49.000	USER_1	LDN	PAI
> 2026-06-25 15:28:39.000	USER_1	NYC	LDI
> 2026-06-25 15:28:23.000	USER_1	PAR	NY
> 2026-06-25 15:28:04.000	USER_1	Online Gateway	PAI

Part 2: Formatting visuals

1. Select velocity attack visual & click Edit button
2. Do the following changes
 - a. Name: Live Velocity Attacks
 - b. Visual type: Bar chart
 - c. **X-axis:** cust_id
 - d. **Y-axis:** total_spend

Visual type

Bar chart

Visual format

Bar chart

General

Tile name

Live Velocity Attacks

Hide tile name
☐

Tile description

Insert description

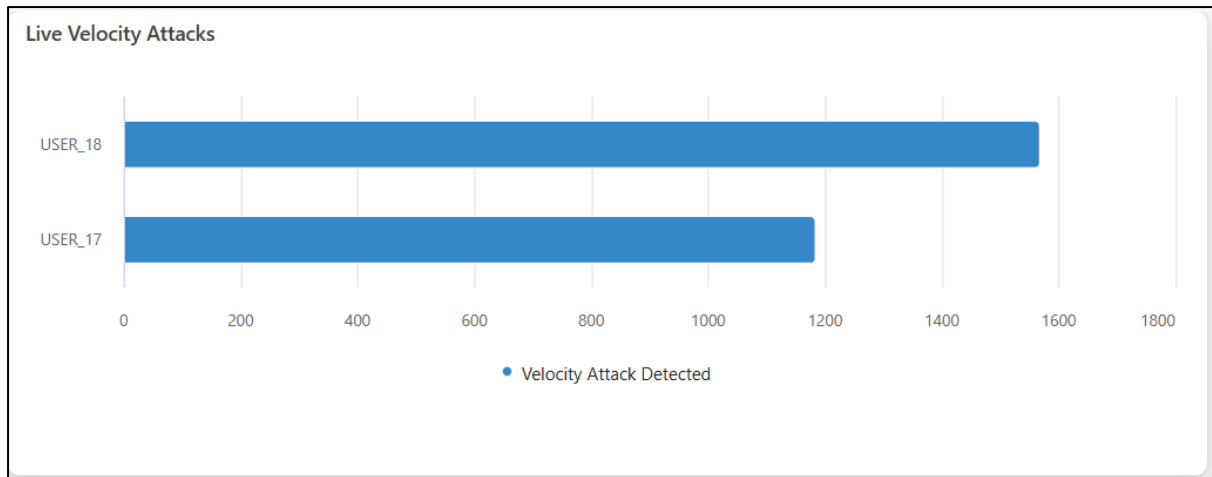
Data

Y columns

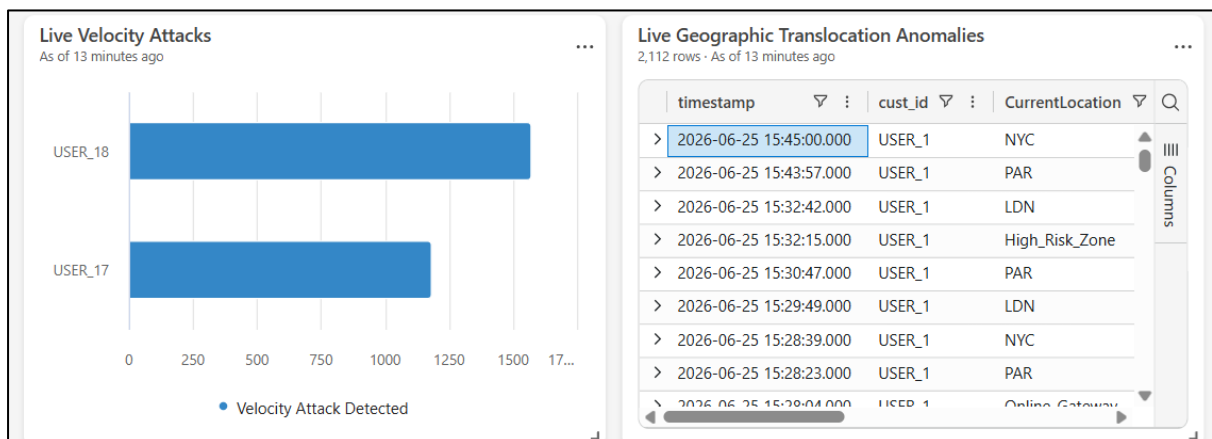
total_spend (real)

X column

cust_id (string)

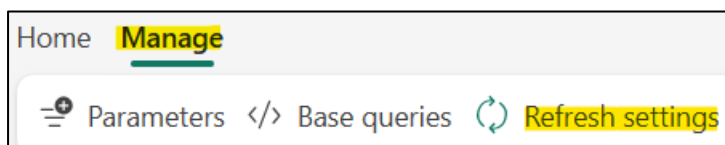


3. Click **Done**
4. Select the next tile
5. Do the following changes
 - a. Name: Live Geographic Translocation Anomalies
6. Click **Done**



Part 3: Enabling the Continuous Stream

1. Click on Manage -> Refresh settings



Refresh settings popup is displayed

Refresh settings

☐ Live refresh (Recommended)

Dashboards visuals will be refreshed automatically once newly ingested data is added into the data source

Settings

☒ Manual refresh only

Dashboards visuals will remain static until viewers manually refresh them

[Learn more](#)

Apply

Cancel

2. Select Live refresh and click Apply
3. If the script is running, the visuals should refresh automatically

Enhancements

1) Implement a 30-Day Cold Storage Policy (Data Lifecycle Management). Clue: `.alter table RawTransactions policy retention`

2) Create a "Top 5 High-Risk Cities" query and create a pie chart visual in the dashboard

RawTransactions

| where fraud_type != "None"

| summarize IncidentCount = count() by location

| top 5 by IncidentCount desc

3) Create additional visuals in Power BI report

4) Multi-Channel Alerting via Webhooks (Data Activator) : configure the Reflex item to call a custom **Webhook** that posts an alert message directly into a Microsoft Teams/Slack channel, or targets a mock REST API endpoint designed to programmatically freeze a bank account